Embedded Linux & System Software Interview Handbook

Chapter 7 - Linux Kernel Synchronization & Concurrency (Part 3)

Topics: Mutexes, Semaphores and Completions

7.4 Mutexes

A mutex (mutual exclusion lock) protects shared resources in process context. Unlike a spinlock, a thread waiting for a mutex sleeps instead of consuming CPU cycles. Therefore, mutexes are suitable for long critical sections or operations that may block.

When to Use a Mutex

• File operations
• User-space requests
• Long critical sections
• Operations that may sleep (msleep(), schedule(), I/O)

Mutex APIs

API	Purpose
mutex_init()	Initialize a mutex
mutex_lock()	Acquire the mutex
mutex_unlock()	Release the mutex
mutex_trylock()	Non-blocking lock attempt

7.5 Semaphores

A semaphore controls access to a limited number of shared resources. Binary semaphores allow one owner, while counting semaphores permit multiple concurrent users.

Semaphore APIs

• sema_init()
• down()
• down_interruptible()
• up()

7.6 Completions

Completions provide one-time synchronization between threads or between interrupt and process contexts. A waiting thread sleeps until another execution context signals completion.

Completion APIs

• init_completion()
• wait_for_completion()
• complete()
• complete_all()

Mutex vs Spinlock

Feature	Spinlock	Mutex
Can Sleep	No	Yes
Busy Wait	Yes	No
Interrupt Context	Yes	No
Typical Use	Short critical sections	Long critical sections

Common Mistakes

• Using a mutex in interrupt context.
• Holding a mutex longer than necessary.
• Forgetting to unlock the mutex.
• Using a semaphore when a mutex is more appropriate.

Interview Questions

1. Mutex vs Spinlock?
2. Binary semaphore vs mutex?
3. When should completions be used?
4. Why can't mutexes be used in interrupt context?
5. Explain down() and up().

Key Takeaways

Mutexes are the preferred synchronization primitive for sleeping process context, semaphores manage shared resources, and completions provide simple event synchronization. Selecting the correct primitive improves correctness and performance.